

Programación para Ciencia de Datos

Fundamentos de Ciencia de Datos

Germán A. Montoya O.
gamontoyao@eafit.edu.co

Universidad EAFIT

Medellín 2026

Agenda

- Listas
- Numpy
- Estructuras de Control
- Graficas
- Cadenas, Tuplas y Diccionarios
- Funciones
- Librerías
- Clases

Listas

```
lista=[5,7,2,4,5,8]
```

#Acceder al primer elemento de la lista

```
res1=lista[0]=5
```

#Acceder al segundo elemento de la lista

```
res2=lista[1]=7
```

#Acceder al último elemento de la lista

```
res3=lista[len(lista)-1]=8
```

#Acceder a un subconjunto de datos (slicing). El límite superior no se incluye.

```
res4=lista[1:4]=[7,2,4]
```

Listas

```
lista=[5,7,2,4,5,8]
```

#Devuelve 1 (la posición del 7).

```
res1_1=lista.index(7)=1
```

#Devuelve 2 (el número 5 aparece dos veces).

```
res1_2=lista.count(5)=2
```

Listas

```
lista=[5,7,2,4,5,8]
```

#Acceder a un subconjunto de datos (slicing) llegando hasta el final. El límite superior no se incluye.

```
res5=lista[2:len(lista)]=[2,4,5,8]
```

#Acceder a un subconjunto de datos (slicing) llegando hasta el final.

```
res6=lista[2:]=[2,4,5,8]
```

#Subconjunto desde el principio hasta el 4to elemento de la lista.

```
res7=lista[:4]=[5,7,2,4]
```

Listas

```
lista=[5,7,2,4,5,8]
```

```
#Seleccionar el último elemento de la lista.
```

```
res8=lista[-1]=[8]
```

```
#Seleccionamos los dos últimos elementos de la lista.
```

```
res9=lista[-2:]=[5,8]
```

```
#Seleccionamos los tres últimos elementos de la lista.
```

```
res10=lista[-3:]=[4,5,8]
```

```
#Invertimos la lista
```

```
res10_1=lista[::-1]=[8, 5, 4, 2, 7, 5]
```

Listas

```
lista=[5,7,2,4,5,8]
```

#Inserción al final

```
res10_2=lista.append(10)=[5, 7, 2, 4, 5, 8, 10]
```

#Inserción al final

```
res10_3=lista.insert(1, 99)=[5, 99, 7, 2, 4, 5, 8]
```

#Extender con otra lista

```
res10_4=lista.extend([0, 0])=[5, 7, 2, 4, 5, 8, 0, 0]
```

Listas

```
lista=[5,7,2,4,5,8]
```

#Elimina la primera aparición del 5.

```
lista.remove(5)=[7, 2, 4, 5, 8, 10]
```

#Elimina el elemento en el índice 3 (el 4) y lo guarda en otra variable.

```
res10_6=lista.pop(3)=4
```

#Eliminar el último elemento.

```
res10_7=lista.pop()=[5, 7, 2, 4, 5]
```


Listas

```
lista=[5,7,2,4,5,8]
```

```
#Concatenamos 2 listas.
```

```
res11=[1,2,3]+[4,5]=[1,2,3,4,5]
```

```
#Creamos una lista con la lista [1,2,3] repetida 10 veces.
```

```
res12=[1,2,3]*10
```

```
#Construir una lista que tenga un 1 rodeado de dos listas, cada una con 20 ceros.
```

```
res13=([0]*20)+[1]+([0]*20)
```

```
#Calculamos el promedio de la lista
```

```
res14=sum(lista)/len(lista)
```

```
#Organizamos la lista de menor a mayor.
```

```
res15=sorted(lista)
```

Listas

```
lista=[5,7,2,4,5,8]
```

```
#Ordenar de menor a mayor.
```

```
res15_1=lista.sort()=[2, 4, 5, 5, 7, 8]
```

```
#Ordenar de mayor a menor.
```

```
res15_2=lista.sort(reverse=True)=[8,7,5,5,4,2]
```

```
#Obtenemos el máximo.
```

```
res15_3=max(lista)=8
```

```
#Obtenemos el mínimo.
```

```
res15_4=min(lista)=2
```

Listas

#Elevar a una potencia.

res16=2**5=32

#División entera.

res17=5//2=2

#Operador modulo.

res18=5%2=1

Listas

```
lista=[5,7,2,4,5,8]
```

#Elevar todo al cuadrado (List Comprehension).

```
res19=[x**2 for x in lista]=[25, 49, 4, 16, 25, 64]
```

#Filtrar solo los mayores de 5.

```
res20=[x for x in lista if x > 5]=[7, 8]
```

#Reemplazar todos los 5 por 100.

```
res21=[100 if x == 5 else x for x in lista]=[100, 7, 2, 4, 100, 8]
```

Listas

#Lista de listas

```
matriz = [  
    [1, 2, 3], # Fila 0  
    [4, 5, 6], # Fila 1  
    [7, 8, 9]  # Fila 2  
]
```

#Acceder a una fila completa

```
print(matriz[1]) → devuelve [4, 5, 6]
```

#Acceder a un elemento específico:

```
print(matriz[1][2]) → devuelve 6 (Fila 1, Columna 2)
```

Listas

```
#Recorrer la matriz
```

```
for fila in matriz:  
    for elemento in fila:  
        print(elemento)
```

```
#List Comprehension en 2D
```

```
plana = [elemento for fila in matriz for elemento in fila]
```

```
# Resultado: [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Listas

#No tienen que ser cuadradas

```
irregular = [  
    [1, 2],  
    [3, 4, 5, 6],  
    [7]  
]
```

```
for fila in irregular:  
    print(f"Procesando fila de longitud {len(fila)}")  
    for elemento in fila:  
        print(f" Elemento: {elemento}")
```

Listas

#No tienen que ser cuadradas

```
irregular = [  
    [1, 2],  
    [3, 4, 5, 6],  
    [7]  
]
```

#Usando índices (Si se necesita la posición)

```
for i in range(len(irregular)):  
    for j in range(len(irregular[i])):  
        print(f"En la fila {i}, columna {j} está el: {irregular[i][j]}")
```


Taller 1A

- El taller 1 se compone de varios items (A, B, C, etc).
- Todo el taller 1 se debe realizar de manera individual.
- Implementar todos los ejemplos de las listas.

Librería Numpy

```
import numpy as np
```

```
#Creamos un arreglo a partir de una lista.
```

```
data = [0,2,4,6]
```

```
res2 = np.array(data)
```

```
#Obtenemos la forma (filas y columnas) del arreglo.
```

```
res3=res2.shape=(4,)
```

```
#Obtenemos las filas del arreglo.
```

```
res4=res2.ndim=1
```

```
#Obtenemos el número de elementos del arreglo. Multiplico filas*columnas.
```

```
res5=res2.size=4
```

Librería Numpy

#Creamos un arreglo de 2 filas y 4 columnas a partir de listas.

```
data = [[0, 2, 4, 6], [1, 'a', 5, 7]]  
res8 = np.array(data)
```

#Creamos un arreglo de 2 filas y 4 columnas a partir de tuplas.

```
data = [(0, 2, 4, 6), (1, 'a', 5, 7)]  
res9 = np.array(data)
```

#Creamos un arreglo con filas de distinto tamaño a partir de listas y tuplas.

```
data = [[0, 2, 4, 6], (1, 'a', 5, 7, 9)]  
res10 = np.array(data)
```

Librería Numpy

#Accedemos al ultimo elemento de la 2da fila.

```
res11=res10[1][4]=9
```

#Creamos un arreglo a partir de un limite inferior, limite superior e incremento del arreglo.

```
res12 = np.arange(2, 10, 3)=[2,5,8]
```

#Creamos un arreglo a partir de un range con limite inferior, limite superior y número de elementos del arreglo.

```
res13 = np.linspace(2, 10, 4)=[2,4.666,7.333,10]
```

#Creamos una matriz de ceros de 3x3.

```
res14=np.zeros((3,3))
```

Librería Numpy

#Creamos una matriz de unos de 3x3.
`res15=np.ones((3,3))`

#Extraemos la 2da fila.
`res16=res15[1,:]`

#Extraemos la 2da columna.
`res17=res15[:,1]`

Librería Numpy

#Indexado booleano

```
edades = np.array([23, 56, 67, 89, 23, 56, 27, 12, 8, 72])
```

```
res18=edades > 30=[False True True...True]
```

```
generos = np.array(['m', 'm', 'f', 'f', 'm', 'f', 'm', 'm', 'm', 'f'])
```

```
res19=generos == 'm'=[True True False ...False]
```

```
res20=(edades > 10) & (generos == 'f')=[False False True ...  
True]
```

Librería Numpy

#Indexado booleano

```
edades = np.array([23, 56, 67, 89, 23, 56, 27, 12, 8, 72])  
res18=edades > 30=[False True True...True]
```

```
generos = np.array(['m', 'm', 'f', 'f', 'm', 'f', 'm', 'm', 'm', 'f'])  
res19=generos == 'm'=[True True False ...False]
```

#Indexado booleano con extracción de los 'trues'.

```
positivos = (generos == 'm')  
res21=edades[positivos]=[23,56,23,27,12,8]
```

```
res22=edades[generos == 'm']=[23,56,23,27,12,8]
```

Librería Numpy

#Transpuesta de una matriz.

```
data = [[0, 2, 4, 6], [1, 'a', 5, 7]]
```

```
dataArray = np.array(data)
```

```
res23=dataArray.T
```

#Cambiamos las dimensiones de una matriz.

```
res24=dataArray.reshape(4,2)
```

#Convertimos un arreglo de cualquier número de dimensiones en uno de una sola dimensión

```
res25=dataArray.ravel()
```


Librería Numpy

```
edades = np.array([23, 56, 67, 89, 23, 56, 27, 12, 8, 72])
```

```
#Calculamos el mínimo valor del arreglo.  
res26=edades.min()
```

```
#Calculamos el máximo valor del arreglo.  
res27=edades.max()
```

```
#Calculamos el promedio del arreglo.  
res28=edades.mean()
```

```
#Calculamos la varianza del arreglo.  
res29=edades.var()
```

Librería Numpy

```
edades = np.array([23, 56, 67, 89, 23, 56, 27, 12, 8, 72])
```

```
#Calculamos la desviación estándar del arreglo.  
res30=edades.std()
```

```
#Calculamos la frecuencia de cada elemento del arreglo.  
res31=np.bincount(edades)
```

```
#Calculamos la suma acumulada del arreglo.  
res32=edades.cumsum()
```

```
#Calculamos el producto acumulado del arreglo.  
res33=edades.cumprod()
```

Taller 1B

- El taller 1 se compone de varios items (A, B, C, etc).
- Todo el taller 1 se debe realizar de manera individual.
- Implementar todos los ejemplos de Numpy.

Estructuras de Control

#Estructura del If

#operadores relacionales: ==, !=, not, and, or.

x=1

y=2

if x < y:

 print(x, "es menor que", y)

elif x > y:

 print(x, "es mayor que", y)

else:

 print(x, "y", y, "son iguales")

Estructuras de Control

#Ciclo for

#Recorrer una lista mostrando cada elemento.

```
lista=[3, 9, 12, 4]
```

```
for elemento in lista:  
    print(elemento)
```

#Llenar una lista con los elementos de un range. Range(x) genera elementos desde 0 hasta x-1.

```
lista=[]
```

```
for i in range(5):  
    lista.append(i)  
res1=lista
```

Estructuras de Control

#Imprimimos los tres primeros elementos

```
lista = [3, 6, 9, 11]
```

```
i = 0
```

```
while i < 3:
```

```
    print(lista[i])
```

```
    #i = i + 1
```

```
    i += 1
```

#Es posible usar 'break' y 'continue' dentro de bucles.

Taller 1C

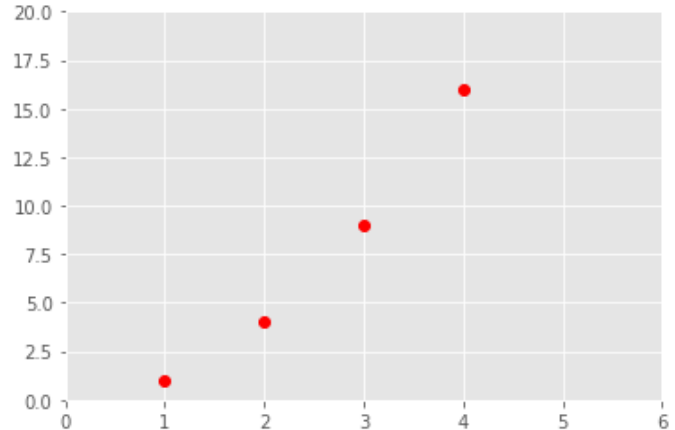
- El taller 1 se compone de varios items (A, B, C, etc).
- Todo el taller 1 se debe realizar de manera individual.
- Implementar todos los ejemplos de las estructuras de control.

Gráficas

```
# Cargamos la librería  
import matplotlib.pyplot as plt  
plt.style.use('ggplot')
```

Pintamos una señal donde la primera lista es el eje del tiempo y las segunda lista son los valores de la función.

```
plt.plot([1,2,3,4], [1,4,9,16], 'ro')  
plt.axis([0, 6, 0, 20])  
plt.show()
```



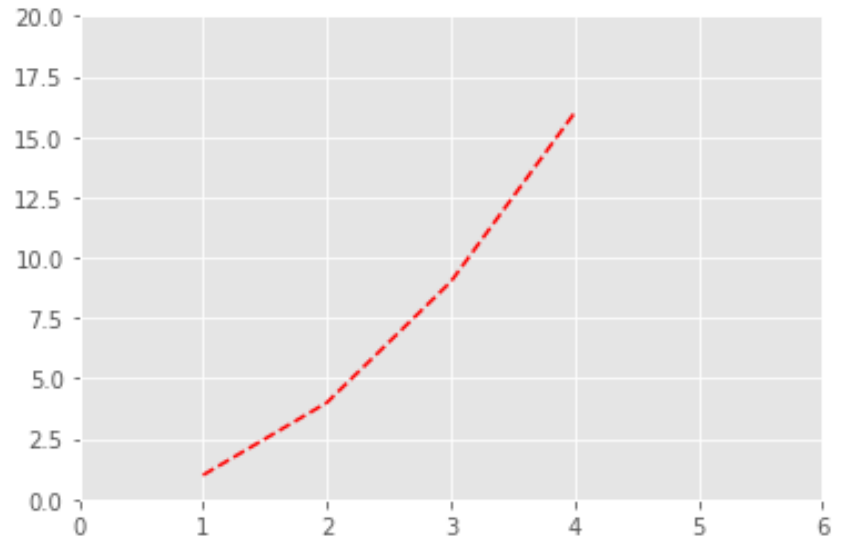
Gráficas

Cuando queremos que las líneas sean interlineadas.

```
plt.plot([1,2,3,4], [1,4,9,16], 'r--')
```

```
plt.axis([0, 6, 0, 20])
```

```
plt.show()
```



Gráficas

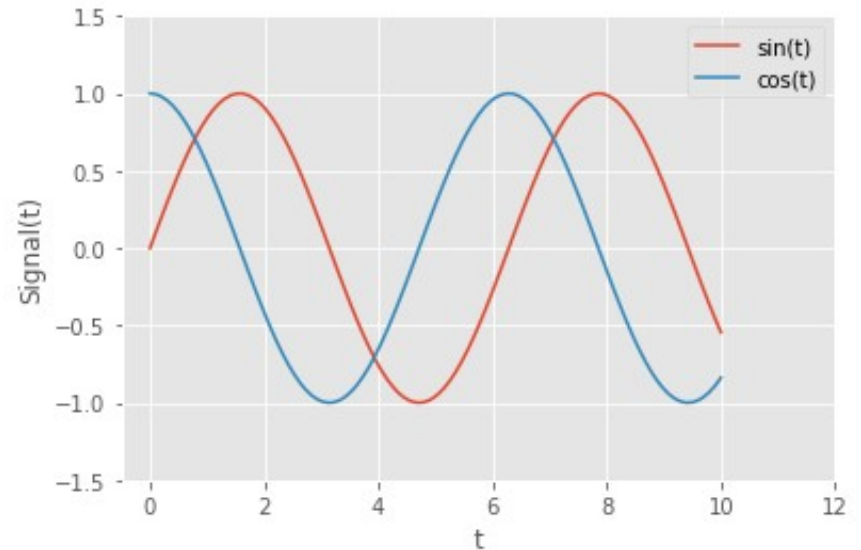
Cuando queremos graficar una señal senoidal en el eje del tiempo.

```
t = np.linspace(0, 10.0, 100)
plt.plot(t, np.sin(t))
plt.xlabel('Time')
plt.ylabel('Signal')
plt.title('My Plot  $\frac{a}{b}$ ');
plt.show()
```



Gráficas

```
# Varias curvas en la misma figura.  
t = np.linspace(0, 10.0, 100)  
plt.figure(1)  
plt.plot(t, np.sin(t), label='sin(t)')  
plt.plot(t, np.cos(t), label='cos(t)')  
plt.xlabel('t')  
plt.ylabel('Signal(t)')  
plt.ylim(-1.5, 1.5)  
plt.xlim(right=12.0)  
plt.legend()  
plt.show()
```



Gráficas

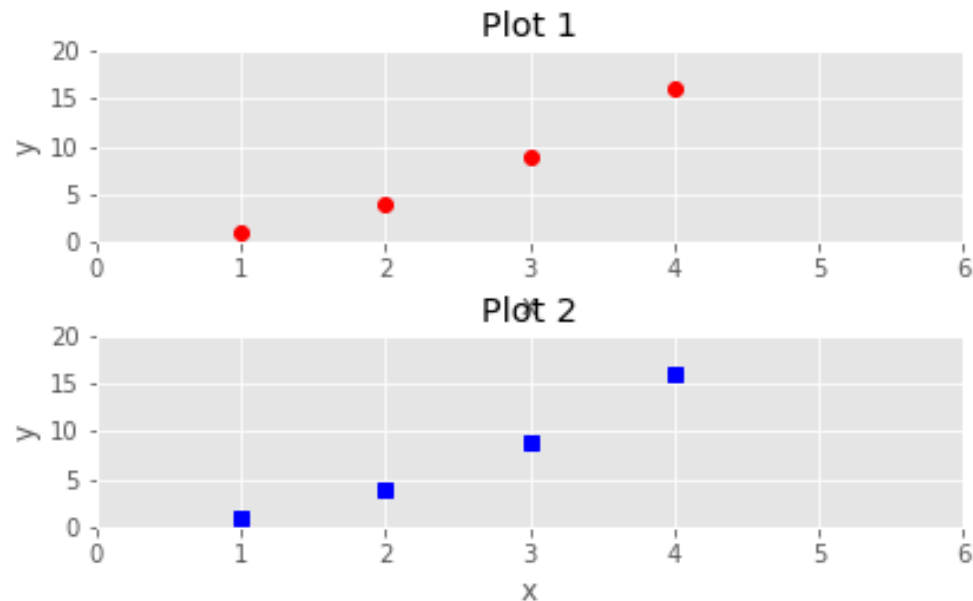
Cuando queremos manejar subplots (varias curvas en la misma figura pero separadas).

```
plt.figure(1)
plt.subplot(211)
plt.plot([1,2,3,4], [1,4,9,16], 'ro')
plt.xlabel('x')
plt.ylabel('y')
plt.title('Plot 1')
plt.axis([0, 6, 0, 20])
plt.grid(True)
```

```
plt.subplot(212)
plt.plot([1,2,3,4], [1,4,9,16], 'bs')
plt.xlabel('x')
plt.ylabel('y')
plt.title('Plot 2')
plt.axis([0, 6, 0, 20])
plt.grid(True)
```

```
plt.subplots_adjust(top=0.9, bottom=0.2, left=0.10, right=0.95, hspace=0.5, wspace=1)
```

```
plt.show()
```



Gráficas

Poner un texto en la curva.

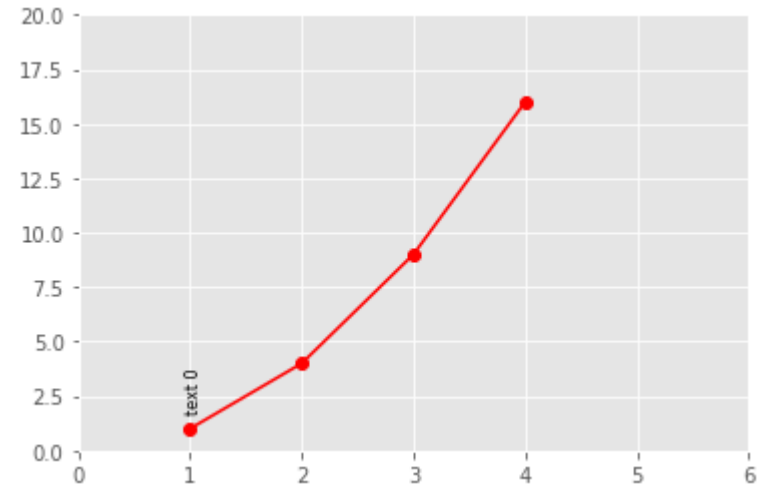
```
plt.figure(1)
```

```
plt.plot([1,2,3,4], [1,4,9,16], 'ro-')
```

```
plt.axis([0, 6, 0, 20])
```

```
plt.text(0.95, 3.2, 'text 0', rotation=90, size=8)
```

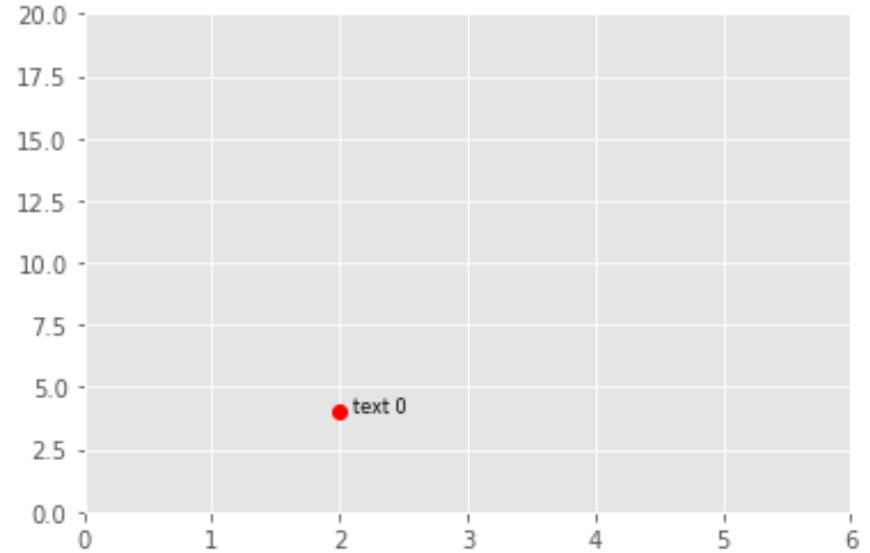
```
plt.show()
```



Gráficas

Graficar una única coordenada y escribiendo un texto.

```
plt.figure(1)
plt.plot([2], [4], 'ro-')
plt.axis([0, 6, 0, 20])
plt.text(2.1, 4, 'text 0', size=8)
plt.show()
```



Taller 1D

- El taller 1 se compone de varios items (A, B, C, etc).
- Todo el taller 1 se debe realizar de manera individual.
- Implementar todos los ejemplos de las gráficas.

Cadenas

```
#Cadena de caracteres
```

```
fruta='banano'
```

```
dulce='bocadillo'
```

```
#Concatenamos cadenas sin un espacio intermedio.
```

```
res1=fruta+dulce
```

```
#Concatenamos cadenas (con espacio).
```

```
res2=fruta + ' ' + dulce
```

```
#Extraemos 'ban' de res2.
```

```
res3=res2[0:3]
```


Cadenas

#Convertimos la cadena en mayúsculas.

```
res4=fruta.upper()
```

#Contamos cuantas veces se repite un caracter.

```
res5=fruta.count('n')
```

#NO es posible modificar un caracter de una cadena usando la asignación empleada en las listas.

```
#fruta[0]='a'
```

#Sin embargo, es posible reemplazar caracteres por otros.

```
res6=fruta.replace('n','l')
```

Cadenas

#Reemplazar grupos de caracteres por otros.

```
res7=fruta.replace('ana','er')
```

#Dividimos una cadena en palabras.

```
res8=res2.split()
```

#Dividimos una cadena en palabras con base en un carácter específico.

```
res9=res2.split('a')=['b', 'n', 'no boc', 'dillo']
```

Tuplas

#Definimos una tupla. No podemos modificar sus valores.

```
tupla=(5,7,'b',0.1)
```

#Definimos una tupla. No podemos modificar sus valores. Podemos omitir los paréntesis.

```
tupla=5,7,6,0.1
```

#Definimos una tupla de un solo elemento.

```
tupla2=(1,)
```

#Accedemos al 3er elemento.

```
res11=tupla[2]
```

Tuplas

#Extraemos un subconjunto.

```
res12=tupla[1:3]
```

#Extraemos el último elemento.

```
res13=tupla[-1]
```

#Extraemos los dos últimos elementos.

```
Res14=tupla[-2:]
```

#Sacamos el promedio de la tupla.

```
res15=sum(tupla)/len(tupla)
```

#Ordenamos la tupla de menor a mayor.

```
res20=sorted(tupla)
```

Diccionarios

#Creamos un diccionario.

```
codigos = {'Luis': 2257, 'Juan': 9739, 'Carlos': 5591}
```

#Extraemos un elemento.

```
res21=codigos['Juan']=9739
```

#Los elementos de un diccionario se pueden modificar.

```
codigos['Juan']='0000'
```

#Eliminamos un elemento del diccionario.

```
codigos.pop('Juan')
```

#Agregamos un elemento del diccionario.

```
codigos['Jorge'] = 6621
```

Diccionarios

#Extraemos todas las llaves del diccionario.

```
res22=codigos.keys()
```

#Extraemos todos los valores del diccionario.

```
res23=codigos.values()
```

#Para saber si una persona ya está en el diccionario.

```
res24='Carlos' in codigos.keys()
```

#Para saber si un valor ya está en el diccionario.

```
res25=2257 in codigos.values()
```

#Convierte una lista de listas, donde cada una tiene dos elementos, a un diccionario.

```
res26=dict([[10, 'a'], [15, 't']])={10: 'a', 15: 't'}
```

Diccionarios

#Otra manera de crear diccionarios

```
codigos= {}
```

```
codigos["luis"] = [2257]
```

```
codigos["Juan"] = [9739, 7896]
```

```
codigos["Carlos"] = [5591]
```

Taller 1E

- El taller 1 se compone de varios items (A, B, C, etc).
- Todo el taller 1 se debe realizar de manera individual.
- Implementar todos los ejemplos de las cadenas, tuplas y diccionarios.

Funciones

```
# Variable fuera de la función  
mis_numeros = [10, 20, 30]
```

```
def triplicar_valores(lista):  
    for i in range(len(lista)):  
        lista[i] = lista[i] * 3  
    # No hay return aquí
```

```
# Llamamos a la función  
triplicar_valores(mis_numeros)
```

```
print(mis_numeros)  
# Resultado: [30, 60, 90]
```

Funciones

```
# Variable original
```

```
mis_numeros = [10, 20, 30]
```

```
def obtener_triplicados(lista):
```

```
    # Creamos una lista nueva (por ejemplo, con list comprehension)
```

```
    nueva_lista = [x * 3 for x in lista]
```

```
    return nueva_lista
```

```
resultados_nuevos = obtener_triplicados(mis_numeros)
```

```
print("Original:", mis_numeros)    # [10, 20, 30]
```

```
print("Nueva:", resultados_nuevos)  # [30, 60, 90]
```

Funciones

#Definición de una función con varios parámetros de entrada y salida.

```
def operaciones(x,y):
```

```
    w=x+y
```

```
    z=x**y
```

```
    return w,z
```

```
res3=operaciones(2,3)
```

#Llamar una función desde otro archivo.

```
from nombreArchivoFuncion import *
```

```
res3=operaciones(2,3)
```

Taller 1F

- El taller 1 se compone de varios items (A, B, C, etc).
- Todo el taller 1 se debe realizar de manera individual.
- Implementar todos los ejemplos de las funciones.

Clases

#EJEMPLO 1:

#Definimos una clase punto2D la cuál asigna al objeto 2 campos: coordenada en X y coordenada en Y.

```
class punto2D:
```

```
    def __init__(self, coorX, coorY):
```

```
        self.coorX = coorX
```

```
        self.coorY = coorY
```

```
p=punto2D(10,20)
```

#Podemos modificar algún campo de p.

```
p.coorX=30
```

Clases

#EJEMPLO 2:

#Definimos una clase punto2D la cuál asigna al objeto 2 campos: coordenada en X y coordenada en Y.

class punto2D:

def __init__(self, coorX, coorY):

self.coorX = coorX

self.coorY = coorY

#Método de la clase para calcular la distancia desde el punto p hasta el origen. No contiene parámetros de entrada.

def distanciaOrigen(self):

return (self.coorX**2 + self.coorY**2)**(1/2)

p=punto2D(10,20)

print(p.distanciaOrigen())

Clases

#EJEMPLO 3:

#Definimos una clase punto2D la cuál asigna al objeto 2 campos: coordenada en X y coordenada en Y.

class punto2D:

def __init__(self, coorX, coorY):

self.coorX = coorX

self.coorY = coorY

#Método de la clase para calcular la distancia desde el punto p hasta el origen. No contiene parámetros de entrada.

def distanciaOrigen(self):

return (self.coorX**2 + self.coorY**2)**(1/2)

#Método de la clase para calcular la distancia desde el punto p hasta otro punto q, el cuál es un parámetro de entrada.

def distanciaPuntos(self, p2):

return ((self.coorX-p2.coorX)**2 + (self.coorY-p2.coorY)**2)**(1/2)

p=punto2D(10,20)

q=punto2D(30,40)

print(p.distanciaOrigen())

print(p.distanciaPuntos(q))

Librerías

#Importamos la librería de funciones matemáticas y usamos una función (función 'sin').

```
import math
```

```
res1=math.sin(math.pi/2)
```

#Importamos directamente una función específica de la librería de funciones matemáticas.

```
from math import sin, pi
```

```
res2=sin(pi/2)
```


Librerías

#Importamos todas las funciones de la librería.

```
from math import *
```

```
res3=sin(pi/2)
```

#Importamos la librería y le asignamos un nombre abreviado.

```
import math as mt
```

```
res4=mt.sin(mt.pi/2)
```

Librerías

#Ejemplo de instalación de cualquier librería.

#Es solo un ejemplo, pandas ya viene precargado en Google Colab.

!pip install pandas

Nota: muchas librerías de ciencia de datos ya están precargadas en Google Colab, por ejemplo: numpy, matplotlib, pandas, scikit-learn, tensorflow, keras, entre otras.

Taller 1G

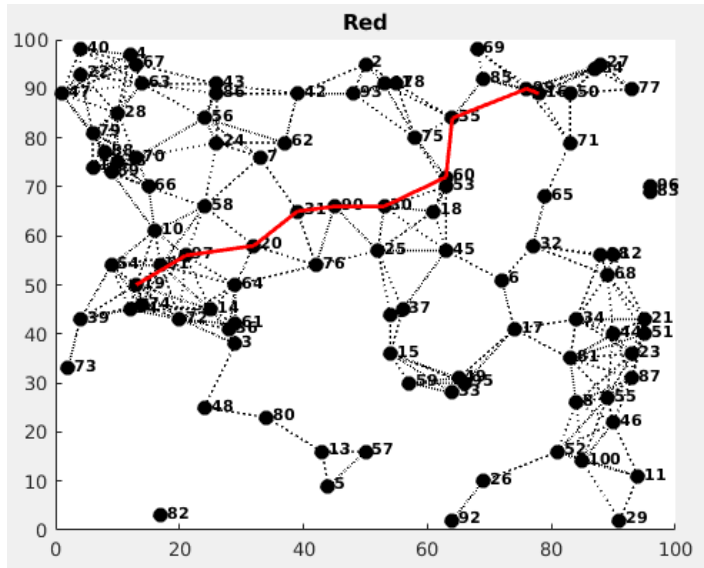
- El taller 1 se compone de varios items (A, B, C, etc).
- Todo el taller 1 se debe realizar de manera individual.
- Implementar todos los ejemplos de las clases y librerías.

Taller 2

- Grafique 4 curvas en ventanas separadas (por ejemplo, 4 curvas senoidales de diferente frecuencia). Recuerde crear un vector t que represente el eje del tiempo (coordenada x). Adiciónale atributos a cada curva (por ejemplo, título de la gráfica, nombre del eje x , nombre del eje y , entre otros).
- Realice el mismo paso anterior, pero use una única ventana, sobreponiendo las curvas.
- Realice el mismo paso 2, pero en una única ventana, graficando cada curva por separado (use la función 'subplot').

Taller 2

- Realice en Python un programa que dibuje 100 nodos cada uno con posiciones aleatorias en un área de 100mX100m con un radio de comunicaciones de 14m y hallando la ruta de mínimo costo entre un nodo fuente y destino aleatorios.



Nota:

- El costo de cada enlace (cuando existe enlace) sería la distancia entre el par de nodos correspondiente.
- Se debe garantizar que el nodo fuente y el nodo destino sean distintos.
- Busque una función que implemente el algoritmo de Dijkstra para encontrar la ruta de mínimo costo entre un nodo fuente y un destino.
- Dibuje los nodos, los números de los nodos, los enlaces de toda la red y la ruta arrojada por el algoritmo de Dijkstra.
- Busque las funciones que sean necesarias para lograr lo que pide el taller.